

Работа с графикой

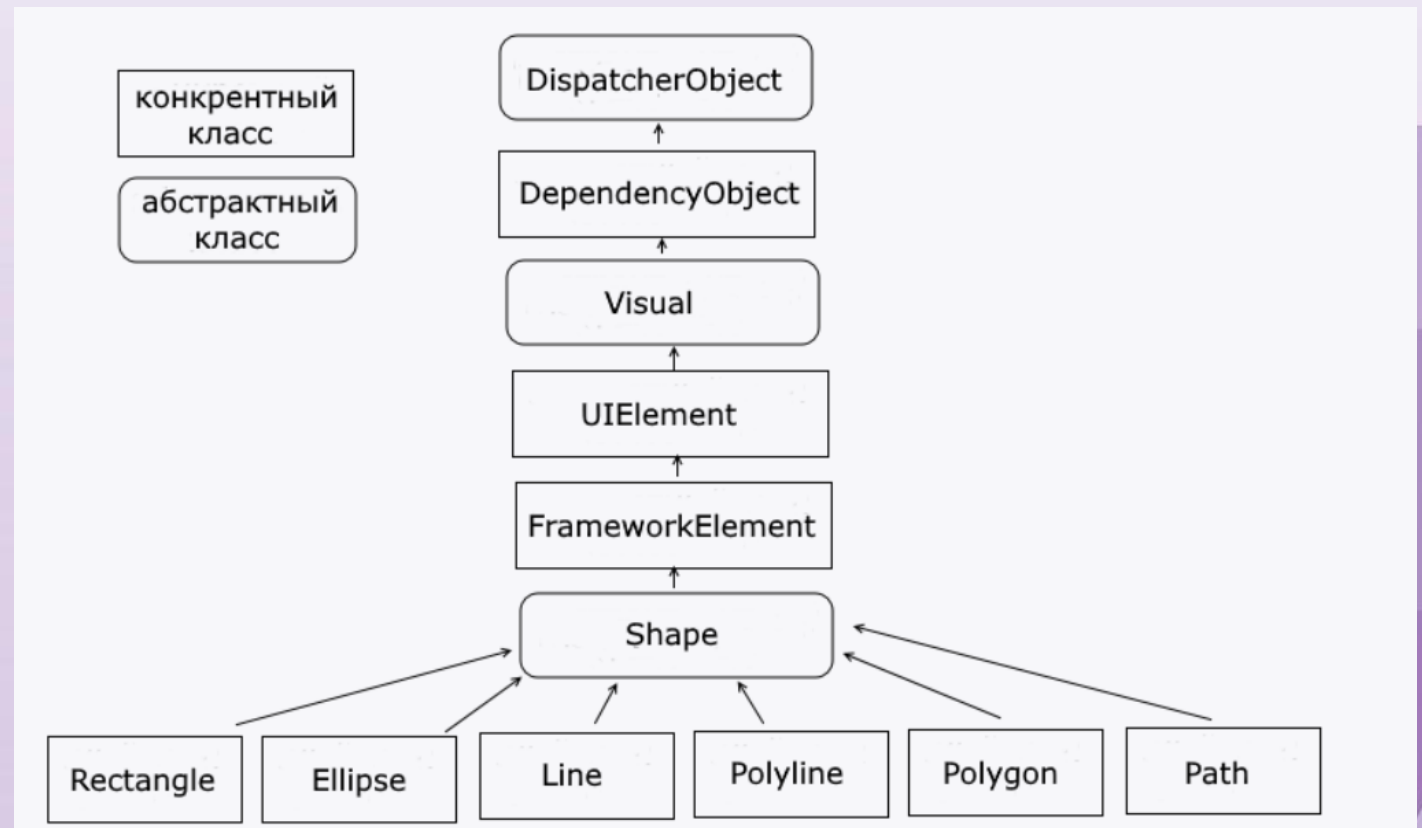
Фигуры

Фигуры

Одним из способов построения двумерной графики в окне - это использование фигур. Фигуры фактически являются обычными элементами как например кнопка или текстовое поле.

К фигурам относят такие элементы как **Polygon** (Многоугольник), **Ellipse** (овал), **Rectangle** (прямоугольник), **Line** (обычная линия), **Polyline** (несколько связанных линий). Все они наследуются от абстрактного базового класса

System.Windows.Shapes.Shape:



Свойства

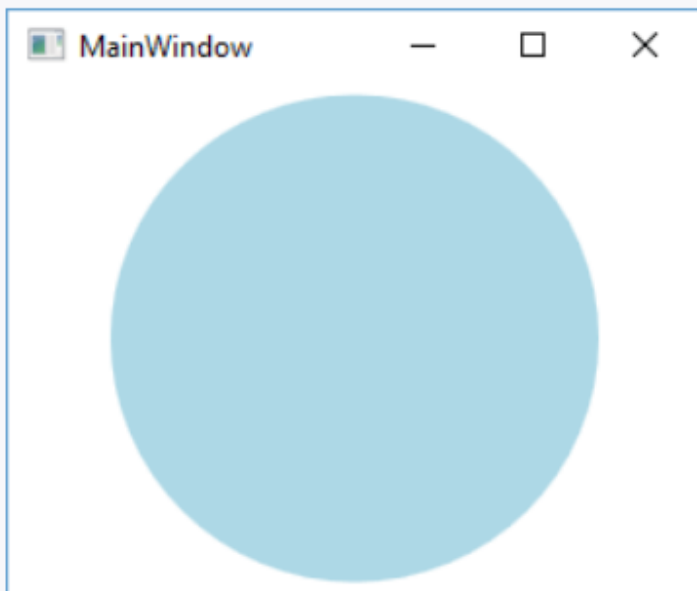
- **Fill** заполняет фон фигуры с помощью кисти - аналогичен свойству **Background** у прочих элементов
- **Stroke** задает кисть, которая отрисовывает границу фигуры - аналогичен свойству **BorderBrush** у прочих элементов
- **StrokeThikness** задает толщину границы фигуры - аналогичен свойству **BorderThikness** у прочих элементов
- **StrokeStartLineCap** и **StrokeEndLineCap** задают для незамкнутых фигур (Line) контур в начале и в конце линии соответственно
- **StrokeDashArray** задает границу фигуры в виде штриховки, создавая эффект пунктира
- **StrokeDashOffset** задает расстояние до начала штриха
- **StrokeDashCap** задает форму штрихов

Ellipse

Ellipse представляет овал:

```
1 <Ellipse Fill="LightBlue" width="200" height="200" />
```

При одинаковой ширине и высоте получается круг:



Rectangle

Rectangle представляет прямоугольник::

```
1 <StackPanel Background="White">
2     <Rectangle Fill="LightBlue" Width="200" Height="100" Margin="10" />
3     <Rectangle Fill="LightPink" Width="200" Height="100" RadiusX="15" RadiusY="15" Margin="10" />
4 </StackPanel>
```

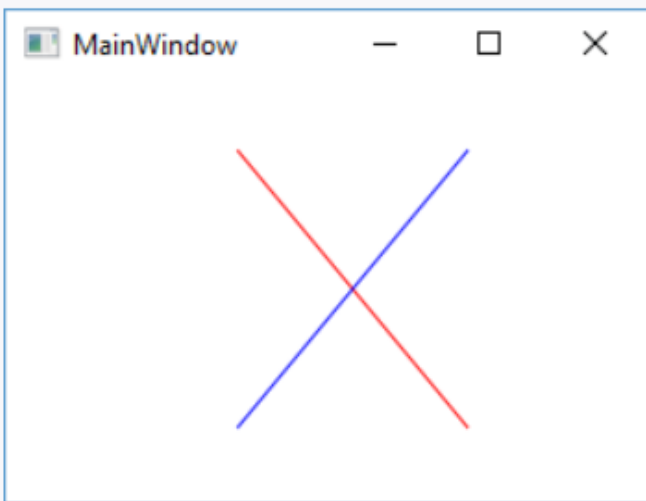
С помощью свойств RadiusX и RadiusY можно округлить углы прямоугольника:



Line

Line представляет простую линию. Для создания линии надо указать координаты в ее свойствах X1, Y1, X2 и Y2. При этом надо учитывать, что началом координатной системы является верхний левый угол:

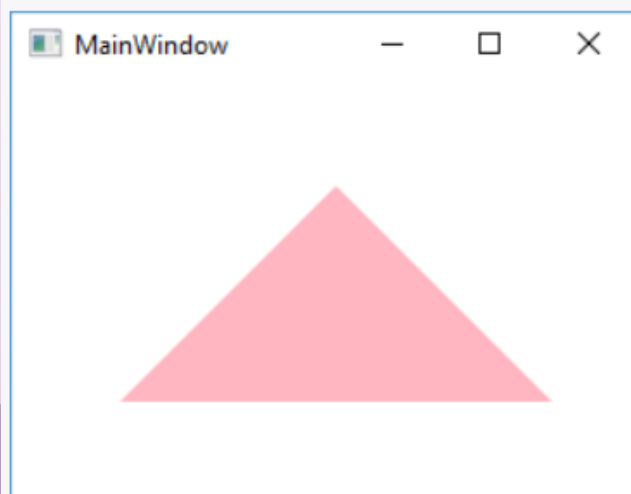
```
1 <Line X1="100" Y1="30" X2="200" Y2="150" Stroke="Red" />  
2 <Line X1="100" Y1="150" X2="200" Y2="30" Stroke="Blue" />
```



Polygon

Polygon представляет многоугольник. С помощью коллекции Points элемент устанавливает набор точек - объектов типа Point, которые последовательно соединяются линиями, причем последняя точка соединяется с первой:

```
1 <Polygon Fill="LightPink" Points="50, 150, 150, 50, 250, 150" />
```



В данном случае у нас три точки (50, 150), (150, 50) и (250, 150), которые образуют треугольник.

Polyline

Polyline представляет набор точек, соединенных линиями. В этом плане данный элемент похож на Polygon за тем исключением, что первая и последняя точка не соединяются:

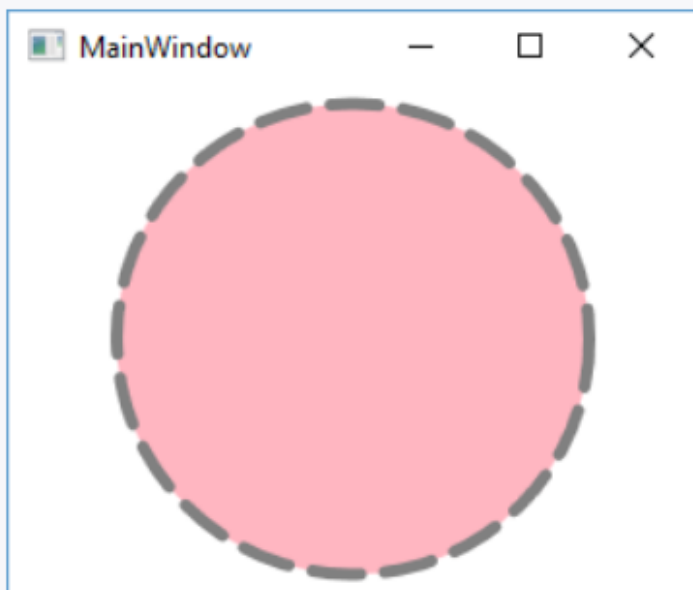
```
1 <Polyline Stroke="Red" Points="50, 150, 150, 50, 250, 150" />
```



Настройка контура

С помощью ряда свойств мы можем настроить отображение контура. Например:

```
1 <Ellipse Width="200" Height="200" Fill="LightPink"
2     StrokeThickness="5" StrokeDashArray="4 2"
3     Stroke="Gray" StrokeDashCap="Round" />
```



Настройка контура

Цвет самого контура определяется с помощью свойства `Stroke`, а его толщина - с помощью **`StrokeThickness`**.

`StrokeDashArray` устанавливает длину штрихов вместе с отступами. Например, `StrokeDashArray="4 2"` устанавливает длину штриха в 4 единицы, а последующего отступа в 2 единицы. И эти значения будут повторяться по всему контуру. При другой установке, например, `StrokeDashArray="1 2 3"` уже задается два штриха. Первый штрих имеет длину в 1 единицу, а второй - в 3 единицы и между ними расстояние в 2 единицы. И так вы можем настроить количество штрихов и расстояний между ними.

`StrokeDashCap` задает форму на концах штрихов и может принимать следующие значения:

- `Flat`: стандартные штрихи с плоскими окончаниями
- `Square`: штрихи с прямоугольными окончаниями
- `Round`: штрихи с округлыми окончаниями
- `Triangle`: штрихи с окончаниями в виде треугольников

Программное рисование

Создание фигур программным образом осуществляется так же, как и создаются и добавляются все остальные элементы:

```
1 Ellipse el = new Ellipse();  
2 el.Width = 50;  
3 el.Height = 50;  
4 el.VerticalAlignment = VerticalAlignment.Top;  
5 el.Fill = Brushes.Green;  
6 el.Stroke = Brushes.Red;  
7 el.StrokeThickness = 3;  
8 grid1.Children.Add(el);
```

Пути и геометрии

Фигуры удобны для создания самых простейших рисунков, дизайна, однако что-то более сложное и комплексное с их помощью сделать труднее. Поэтому для этих целей применяется класс **Path**, который представляет геометрический путь. Он также, как и фигуры, наследуется от класса **Shape**, но может заключать в себе совокупность объединенных фигур. Класс **Path** имеет свойство **Data**, которое определяет объект **Geometry** - геометрический объект для отрисовки. Этот объект задает фигуру или совокупность фигур для отрисовки.

Класс **Geometry** - абстрактный, поэтому в качестве объекта используется один из производных классов:

- **LineGeometry** представляет линию, эквивалент фигуры **Line**
- **RectangleGeometry** представляет прямоугольник, эквивалент фигуры **Rectangle**
- **EllipseGeometry** представляет эллипс, эквивалент фигуры **Ellipse**
- **PathGeometry** представляет путь, образующий сложную геометрическую фигуру из простейших фигур
- **GeometryGroup** создает фигуру, состоящую из нескольких объектов **Geometry**
- **CombinedGeometry** создает фигуру, состоящую из двух объектов **Geometry**
- **StreamGeometry** - специальный объект **Geometry**, предназначенный для сохранения всего геометрического пути в памяти

LineGeometry

Например, использование LineGeometry:

```
1 <Path Stroke="Blue">
2   <Path.Data>
3     <LineGeometry StartPoint="100,30" EndPoint="200,130" />
4   </Path.Data>
5 </Path>
```

будет аналогично следующему объекту Line:

```
1 <Line X1="100" Y1="30" X2="200" Y2="130" Stroke="Blue" />
```

Свойства **StartPoint** и **EndPoint** задают начальную и конечную точки линии.

RectangleGeometry

```
1 <StackPanel>
2     <Path Fill="LightBlue">
3         <Path.Data>
4             <RectangleGeometry Rect="100,20 100,50" />
5         </Path.Data>
6     </Path>
7     <Path Fill="LightPink">
8         <Path.Data>
9             <RectangleGeometry Rect="100,20 100,50" RadiusX="10" RadiusY="10" />
10        </Path.Data>
11    </Path>
12 </StackPanel>
```

Свойство **Rect** задает параметры прямоугольника в формате "координата X, координата Y ширина, высота". Также с помощью свойств **RadiusX** и **RadiusY** можно задать радиус скругления углов прямоугольника.



EllipseGeometry

```
1 <Path Fill="LightPink" Stroke="LightBlue">
2   <Path.Data>
3     <EllipseGeometry RadiusX="50" RadiusY="25" Center="120,70" />
4   </Path.Data>
5 </Path>
```

Свойство `Center` устанавливает центр овала, а свойства `RadiusX` и `RadiusY` - радиусы.

GeometryGroup

GeometryGroup объединяет несколько геометрий:

```
1 <Path Fill="LightPink" Stroke="LightBlue">
2   <Path.Data>
3     <GeometryGroup FillRule="Nonzero">
4       <LineGeometry StartPoint="10,10" EndPoint="220,10" />
5       <EllipseGeometry Center="100,100" RadiusX="50" RadiusY="40" />
6       <RectangleGeometry Rect="120,100 80,20" RadiusX="5" RadiusY="5" />
7     </GeometryGroup>
8   </Path.Data>
9 </Path>
```

Объект GeometryGroup устанавливает свойство **FillRule**. Если оно равно **EvenOdd** (значение по умолчанию), то перекрывающиеся поверхности двух геометрий являются прозрачными. А при значении **FillRule="Nonzero"** (как в данном случае), перекрывающиеся поверхности геометрий будут окрашены также, как и остальные части пути.



CombinedGeometry

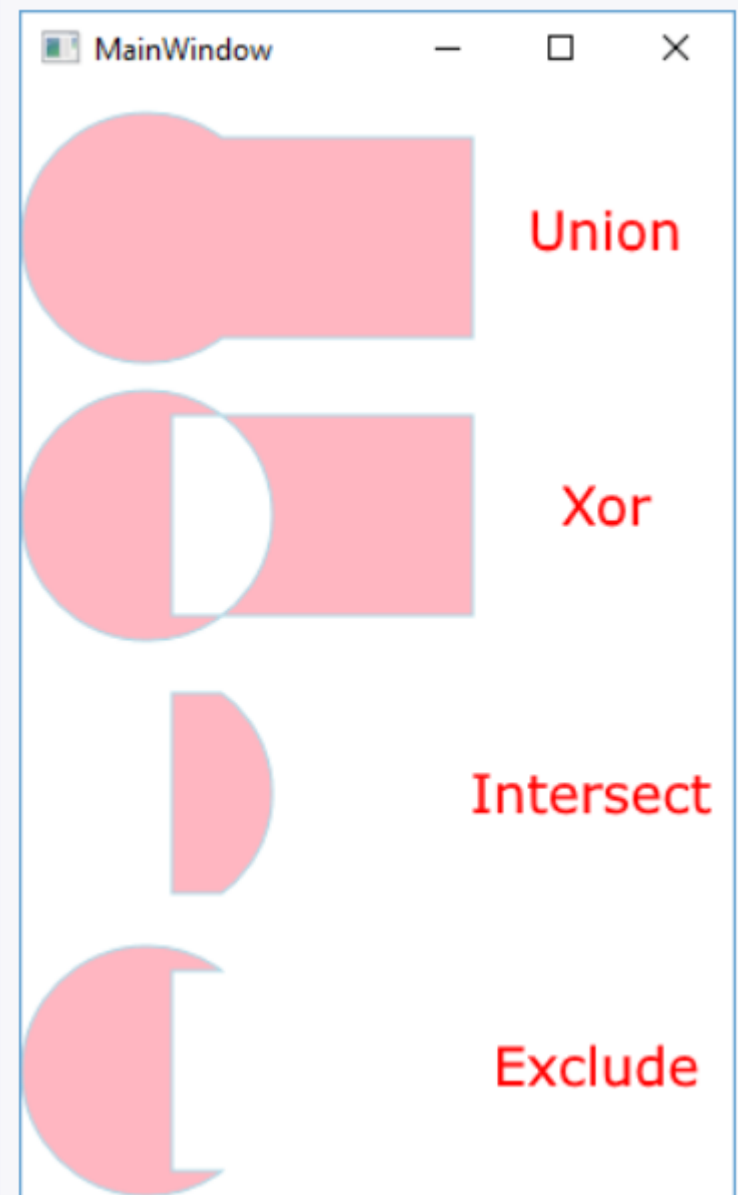
CombinedGeometry состоит из двух геометрий. В этом он похож на GeometryGroup, который также может объединять две геометрии. Однако между ними есть различия. Отличие состоит в том, что объект CombinedGeometry имеет свойство **GeometryCombinedMode**, которое указывает модель перекрытия двух геометрий:

- **Union**: фигура включает обе геометрии
- **Intersect**: фигура включает область, которая одновременно принадлежит обеим геометриям
- **Xor**: фигура включает только непересекающие области геометрий
- **Exclude**: фигура включает первую геометрию с исключением тех областей, которые принадлежат также и второй геометрии

```

1 <Grid>
2   <Grid.RowDefinitions>
3     <RowDefinition />
4     <RowDefinition />
5     <RowDefinition />
6     <RowDefinition />
7 </Grid.RowDefinitions>
8 <Path Fill="LightPink" Stroke="LightBlue">
9   <Path.Data>
10    <CombinedGeometry GeometryCombineMode="Union">
11      <CombinedGeometry.Geometry1>
12        <EllipseGeometry Center="50,60" RadiusX="50" RadiusY="50" />
13      </CombinedGeometry.Geometry1>
14      <CombinedGeometry.Geometry2>
15        <RectangleGeometry Rect="60, 20 120,80" />
16      </CombinedGeometry.Geometry2>
17    </CombinedGeometry>
18  </Path.Data>
19 </Path>
20 <Path Grid.Row="1" Fill="LightPink" Stroke="LightBlue">
21   <Path.Data>
22    <CombinedGeometry GeometryCombineMode="Xor">
23      <CombinedGeometry.Geometry1>
24        <EllipseGeometry Center="50,60" RadiusX="50" RadiusY="50" />
25      </CombinedGeometry.Geometry1>
26      <CombinedGeometry.Geometry2>
27        <RectangleGeometry Rect="60, 20 120,80" />
28      </CombinedGeometry.Geometry2>
29    </CombinedGeometry>
30  </Path.Data>
31 </Path>

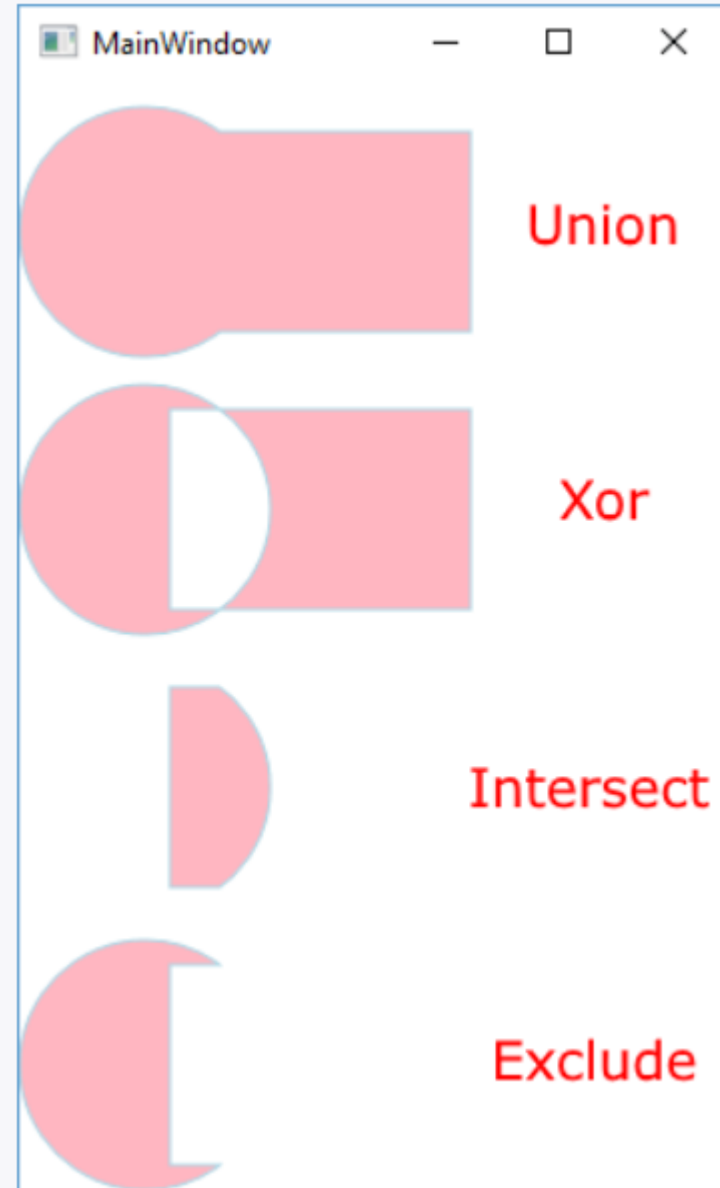
```



```

32 <Path Grid.Row="2" Fill="LightPink" Stroke="LightBlue">
33     <Path.Data>
34         <CombinedGeometry GeometryCombineMode="Intersect">
35             <CombinedGeometry.Geometry1>
36                 <EllipseGeometry Center="50,60" RadiusX="50" RadiusY="50" />
37             </CombinedGeometry.Geometry1>
38             <CombinedGeometry.Geometry2>
39                 <RectangleGeometry Rect="60, 20 120,80" />
40             </CombinedGeometry.Geometry2>
41         </CombinedGeometry>
42     </Path.Data>
43 </Path>
44 <Path Grid.Row="3" Fill="LightPink" Stroke="LightBlue">
45     <Path.Data>
46         <CombinedGeometry GeometryCombineMode="Exclude">
47             <CombinedGeometry.Geometry1>
48                 <EllipseGeometry Center="50,60" RadiusX="50" RadiusY="50" />
49             </CombinedGeometry.Geometry1>
50             <CombinedGeometry.Geometry2>
51                 <RectangleGeometry Rect="60, 20 120,80" />
52             </CombinedGeometry.Geometry2>
53         </CombinedGeometry>
54     </Path.Data>
55 </Path>
56 </Grid>

```



PathGeometry

PathGeometry позволяет создавать более сложные по характеру геометрии. **PathGeometry** содержит один или несколько компонентов **PathFigure**. Объект **PathFigure** в свою очередь формируется из сегментов. Все сегменты наследуются от класса **PathSegment** и бывают нескольких видов:

- **LineSegment** задает отрезок прямой линии между двумя точками
- **ArcSegment** задает дугу
- **BezierSegment** задает кривую Безье
- **QuadraticBezierSegment** задает квадратичную кривую Безье
- **PolyLineSegment** задает сегмент из нескольких линий
- **PolyBezierSegment** задает сегмент из нескольких кривых Безье
- **PolyQuadraticBezierSegment** задает сегмент из нескольких квадратичных кривых Безье

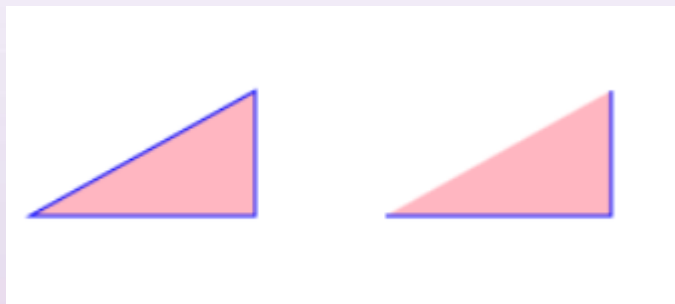
Segment

Эти сегменты составляют свойство **Segment** объекта PathFigure. Кроме того, PathFigure имеет еще несколько важных свойств:

- **StartPoint** - точка начала первой фигуры
- **IsClosed** - если значение равно true, то первая и последняя точки (если они не совпадают) соединяются
- **IsFilled** - если значение равно true, то площадь внутри пути окрашивается кистью, заданной свойством Fill у объекта Path

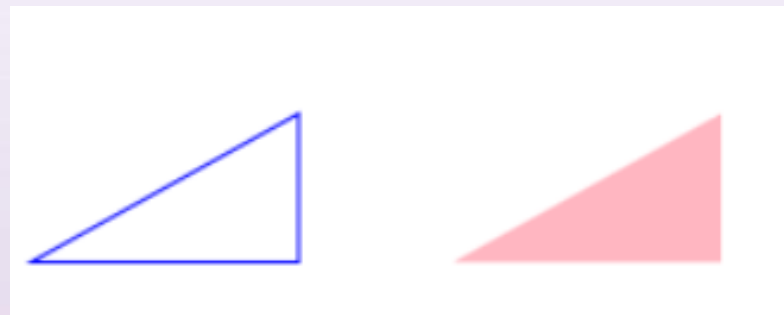
Линии

```
<Path Fill="LightPink" Stroke="Blue">
  <Path.Data>
    <PathGeometry>
      <PathFigure IsClosed="True" StartPoint="10,100">
        <LineSegment Point="100,100" />
        <LineSegment Point="100,50" />
      </PathFigure>
    </PathGeometry>
  </Path.Data>
</Path>
<Path Grid.Column="1" Fill="LightPink" Stroke="Blue">
  <Path.Data>
    <PathGeometry>
      <PathFigure IsClosed="False" StartPoint="10,100">
        <LineSegment Point="100,100" />
        <LineSegment Point="100,50" />
      </PathFigure>
    </PathGeometry>
  </Path.Data>
</Path>
```



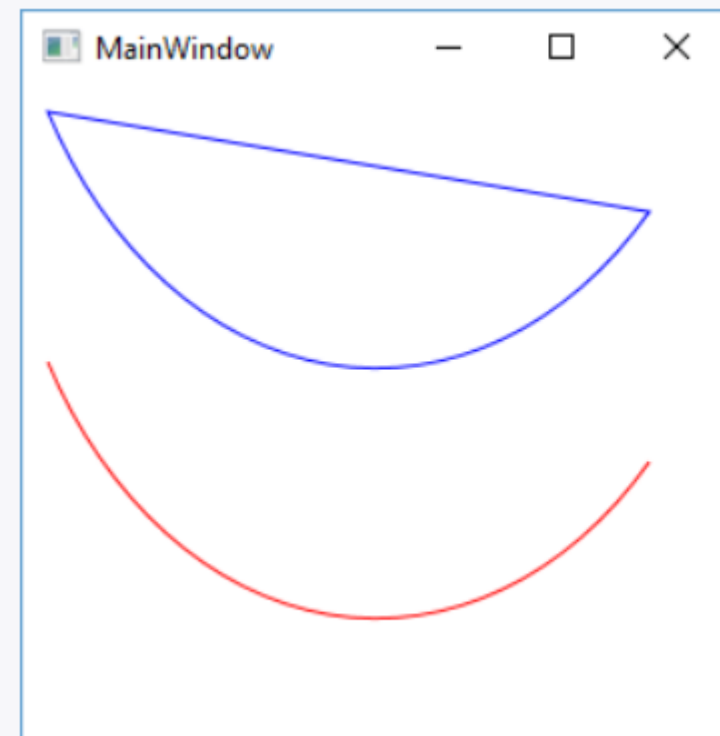
Линии

```
<Path Grid.Row="1" Stroke="Blue">
  <Path.Data>
    <PathGeometry>
      <PathFigure IsClosed="True" StartPoint="10,100">
        <LineSegment Point="100,100" />
        <LineSegment Point="100,50" />
      </PathFigure>
    </PathGeometry>
  </Path.Data>
</Path>
<Path Grid.Row="1" Grid.Column="1" Fill="LightPink">
  <Path.Data>
    <PathGeometry>
      <PathFigure StartPoint="10,100">
        <LineSegment Point="100,100" />
        <LineSegment Point="100,50" />
      </PathFigure>
    </PathGeometry>
  </Path.Data>
</Path>
```



Дуга

```
1 <Grid>
2   <Path Stroke="Blue">
3     <Path.Data>
4       <PathGeometry>
5         <PathFigure IsClosed="True" StartPoint="10,10">
6           <ArcSegment Point="250,50" Size="150,200" />
7         </PathFigure>
8       </PathGeometry>
9     </Path.Data>
10  </Path>
11  <Path Stroke="Red">
12    <Path.Data>
13      <PathGeometry>
14        <PathFigure IsClosed="False" StartPoint="10,110">
15          <ArcSegment Point="250,150" Size="150,200" />
16        </PathFigure>
17      </PathGeometry>
18    </Path.Data>
19  </Path>
20 </Grid>
```



Доп. информация про фигуры

<https://metanit.com/sharp/wpf/17.3.php>

<https://metanit.com/sharp/wpf/17.4.php>

<https://www.c-sharpcorner.com/uploadfile/mahesh/drawing-shapes-in-wpf/>

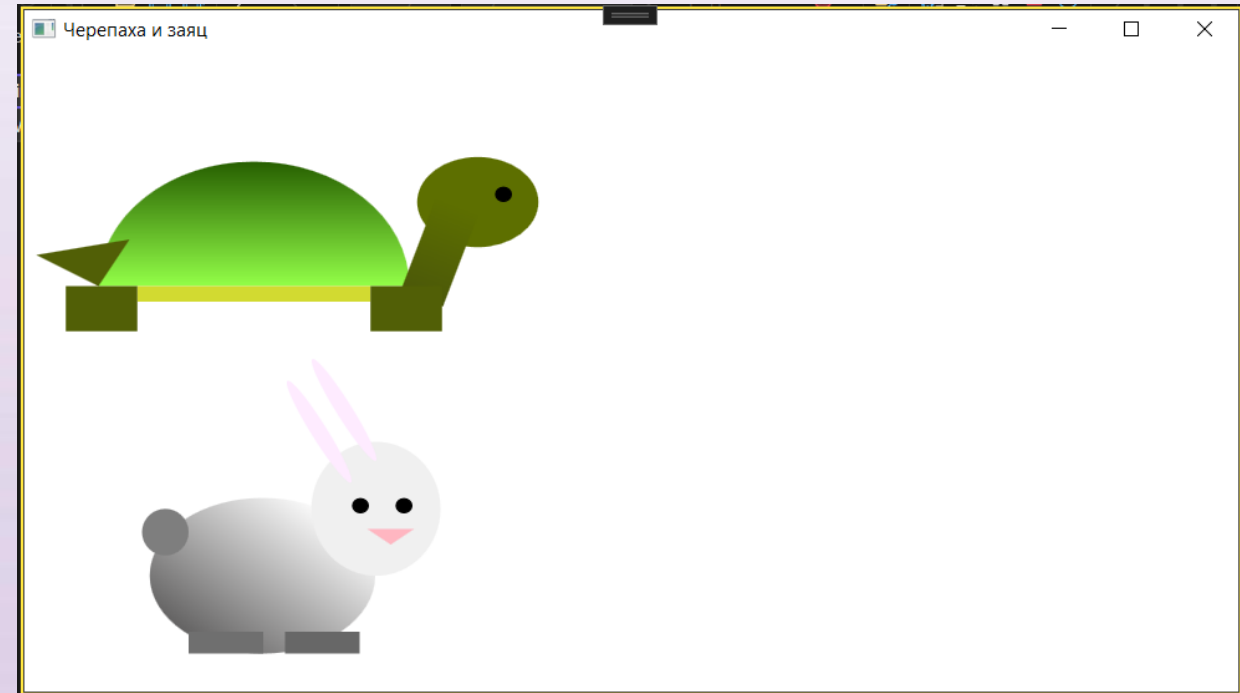
<https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/graphics-multimedia/shapes-and-basic-drawing-in-wpf-overview>

Задание

Создать с помощью фигур двух животных в xaml коде.

Расположите этих животных в начале окна, чтобы при использовании анимации, в будущем, они побежали/попрыгали вперёд.

Используйте компоновку Canvas, он нужен будет для анимации.



```
<Canvas>
  <Canvas x:Name="canvasRubit" Width="300" Height="200" Canvas.Left="90" Canvas.Top="390">
    <Ellipse Height="100" Width="144" Canvas.Left="84" Canvas.Top="50">
      <Ellipse.Fill>
        <LinearGradientBrush EndPoint="0.5,1" StartPoint="0.5,0"...>
      </Ellipse.Fill>
    </Ellipse>
  </Canvas>
```